

# Hotel Management Project - Detailed Student Task And Code Guide

This document explains how to present the project to the teacher. It is divided into exactly 4 students. Each student has a clear section, tasks, code files, and answers to common questions.

## How The Project Runs

1. The browser opens a PHP page inside 'public/', such as 'index.php', 'rooms.php', 'dashboard.php', or an admin page.
2. The page includes 'app/bootstrap.php'.
3. 'bootstrap.php' loads the database configuration, helper functions, session class, models, services, and starts the session.
4. 'Database::connect()' creates a PDO connection to MySQL.
5. The page calls model classes to read or update the database.
6. PHP renders HTML with data from MySQL.
7. JavaScript improves the interface: room modals, photo switching, selected-night preview, and dashboard charts.
8. When the user submits a form, PHP validates the request, updates the database, stores a flash message, and redirects.
9. When the user downloads an invoice, 'InvoiceService' creates a PDF file from reservation and payment data.

## Student 1 - Authentication, Users, Sessions, And Access Control

### Main Responsibility

Student 1 explains how users register, log in, log out, and how the app protects user/admin pages.

### Tasks

- Explain how a user account is created.
- Explain how login checks the email and password.
- Explain how the logged-in user is stored in the PHP session.
- Explain the difference between normal user and admin.
- Explain how protected pages prevent access from guests.
- Explain how admin pages prevent access from normal users.
- Explain default accounts from the database seed.

### Code Files

- 'app/Models/User.php'
- 'app/Services/Auth.php'
- 'app/Core/Session.php'
- 'public/login.php'
- 'public/register.php'
- 'public/logout.php'

- 'public/api/auth.php'
- related helpers in 'app/Core/helpers.php'

## Code Explanation

'User.php' is the model for the 'users' table. It contains methods such as 'all()', 'find()', 'findByEmail()', 'create()', 'update()', 'updatePassword()', and 'delete()'. The model keeps SQL code away from pages.

'Auth.php' is the authentication service. The 'register()' method checks if the email already exists, then creates a user. The 'login()' method finds the user by email, checks the password, and stores user data in the session. The 'logout()' method destroys the session.

'Session.php' wraps PHP session functions. It starts the session, stores values, reads values, deletes values, and handles flash messages. Flash messages are used for success or error messages after redirects.

'login.php' shows the login form. When the form is submitted, it calls 'Auth->login()'. If login succeeds, admins go to '/admin/index.php', and normal users go to '/dashboard.php'.

'register.php' shows the registration form. When submitted, it calls 'Auth->register()' and then redirects to login if successful.

'logout.php' calls 'Auth::logout()' and redirects the user to the home page.

'api/auth.php' exposes login, register, and logout as JSON API actions. This is useful if another frontend wants to talk to the same backend.

## How To Answer The Teacher

If the teacher asks "How does login work?", answer:

"The login page sends email and password to 'Auth::login()'. The Auth service asks the User model to find the user by email. If the password is valid, user information is saved in the PHP session. After that, protected pages can call 'Auth::user()' to know who is logged in."

If the teacher asks "How do you protect admin pages?", answer:

"Every admin page starts with 'Auth::requireAdmin()'. This first checks if the user is logged in, then checks if the role is admin. If not, the user is redirected away."

If the teacher asks "Why use sessions?", answer:

"HTTP is stateless, so without sessions the server forgets the user after each request. Sessions let us remember the logged-in user across pages."

## Student 2 - Hotels, Rooms, Catalog, Room Availability, And Photos

### Main Responsibility

Student 2 explains how hotels and rooms are stored, displayed, grouped, and shown with photos and availability.

### Tasks

- Explain how hotels are created, edited, deleted, and listed.
- Explain how rooms belong to hotels using 'hotel\_id'.

- Explain how the public room catalog groups rooms by country and hotel.
- Explain how availability is calculated and displayed.
- Explain how room and hotel photos are selected.
- Explain the reservation page gallery and night preview.
- Explain admin hotel and room management pages.

## Code Files

- 'app/Models/Hotel.php'
- 'app/Models/Room.php'
- 'app/Core/helpers.php'
- 'public/rooms.php'
- 'public/reserve.php'
- 'public/admin/hotels.php'
- 'public/admin/rooms.php'
- 'public/assets/js/rooms.js'
- 'public/assets/js/reserve.js'
- 'database/schema.sql'

## Code Explanation

'Hotel.php' is the model for the 'hotels' table. It has 'all()', 'find()', 'create()', 'update()', and 'delete()'.

'Room.php' is the model for the 'rooms' table. It also joins hotels and reservations so each room can show hotel name, city, country, stars, photo, and reserved-until date. The repeated room details query is centralized in 'ROOM\_DETAILS\_SELECT'.

'helpers.php' contains display helpers used by the catalog. 'roomTypePhoto()' chooses a representative room photo based on the type, such as suite, family, double, executive, or default. 'roomGallery()' creates a room gallery for the reservation page. 'hotelGallery()' builds hotel preview images. 'roomAvailability()' turns room database status into readable labels like Available, Reserved, or Maintenance.

'rooms.php' loads rooms with '\$roomModel->available()'. It groups them by country and hotel, then prints hotel cards. Each hotel card can show/hide its rooms. Each room card shows type, room number, description, amenities, price, availability, and reserve button.

'reserve.php' loads one room by ID. It shows the main room photo, thumbnails, description, amenities, price, and date form.

'rooms.js' controls hotel card expansion and hotel preview modals.

'reserve.js' changes the main photo when a thumbnail is clicked. It also calculates how many nights are selected from check-in and check-out dates.

'admin/hotels.php' lets the admin create, update, and delete hotels. 'admin/rooms.php' lets the admin create, update, and delete rooms.

'database/schema.sql' creates the 'hotels' and 'rooms' tables and inserts starter hotels and rooms, including real hotels that were added later.

## How To Answer The Teacher

If the teacher asks "How are rooms connected to hotels?", answer:

"The 'rooms' table has a 'hotel\_id' foreign key. In the Room model, SQL joins 'rooms' with

'hotels', so each room can display its hotel name, city, country, and photo."

If the teacher asks "How do photos work?", answer:

"Hotel photos are stored in the 'photo\_url' column. Room photos are selected by helper functions according to the room type. This keeps the UI consistent and avoids repeating image logic in every page."

If the teacher asks "How do you know if a room can be booked?", answer:

"The Room model checks room status and active reservations. Then 'roomAvailability()' converts that data into a message and a boolean 'can\_book'. If 'can\_book' is false, the reserve button is disabled."

## **Student 3 - Reservations, User Dashboard, Support Tickets, And APIs**

### **Main Responsibility**

Student 3 explains how users create reservations, see their dashboard, cancel reservations, ask support questions, and use reservation APIs.

### **Tasks**

- Explain reservation creation.
- Explain date validation.
- Explain total price calculation.
- Explain reservation statuses: pending, confirmed, cancelled.
- Explain user dashboard cards.
- Explain support ticket creation.
- Explain reservation API endpoints.
- Explain why business validation is done in PHP, not only JavaScript.

### **Code Files**

- 'app/Models/Reservation.php'
- 'app/Models/SupportTicket.php'
- 'public/dashboard.php'
- 'public/reserve.php'
- 'public/api/reservations.php'
- 'public/api/response.php'
- 'public/admin/reservations.php'
- 'public/admin/customer-service.php'
- date helpers in 'app/Core/helpers.php'

### **Code Explanation**

'Reservation.php' is the reservation model. It can list all reservations for admin, list reservations for one user, find one reservation, find detailed reservation data, create a reservation, update status, and delete a reservation.

When a reservation is created, 'createAndReturnId()' first checks dates with 'isValidStayRange()'. It rejects invalid dates. Then it loads the room, checks if the room can be reserved, calculates nights using 'nightsBetween()', calculates total price, and inserts the reservation as 'pending'.

'dashboard.php' is the logged-in user dashboard. It shows reservation cards, status, check-in, check-out, total amount, payment button, invoice button, cancellation information, and customer service form.

'SupportTicket.php' stores user questions and admin replies. A user can create a support question from the dashboard. An admin can answer it in 'admin/customer-service.php'.

'api/reservations.php' supports listing reservations, creating reservations, and deleting reservations through JSON endpoints.

'api/response.php' contains helper functions to return JSON and read JSON request bodies.

'admin/reservations.php' lets the admin confirm, cancel, or delete reservations. Confirmation is important because users can only pay after admin confirmation.

## How To Answer The Teacher

If the teacher asks "How is a reservation created?", answer:

"The user chooses dates and submits the form in 'reserve.php'. The Reservation model validates the dates, checks room availability, calculates nights and total price, then inserts a pending reservation into MySQL."

If the teacher asks "Why is the first status pending?", answer:

"Because the admin must confirm the reservation before payment. This gives the hotel control before allowing the user to pay."

If the teacher asks "How is total price calculated?", answer:

"The code calculates the number of nights using 'nightsBetween(check\_in, check\_out)', then multiplies it by the room price."

If the teacher asks "What is the role of the dashboard?", answer:

"The dashboard is the user's control panel. It shows all user reservations, payment state, invoice download, cancellation, and support messages."

## Student 4 - Admin Dashboard, Payments, Invoices, Database, And Final Integration

### Main Responsibility

Student 4 explains the admin panel, payment flow, invoice PDF generation, database connection, schema, and how the project is integrated.

### Tasks

- Explain the admin dashboard statistics.
- Explain the admin CRUD pages.
- Explain how payment works after admin confirmation.
- Explain how the invoice PDF is generated.
- Explain the database connection and schema.
- Explain API stats.
- Explain the shared layout and assets.
- Explain how to run the whole project in XAMPP.

### Code Files

- 'app/Models/Dashboard.php'

- 'app/Models/Payment.php'
- 'app/Services/InvoiceService.php'
- 'config/config.php'
- 'config/Database.php'
- 'app/bootstrap.php'
- 'database/schema.sql'
- 'public/payment.php'
- 'public/invoice.php'
- 'public/admin/index.php'
- 'public/admin/users.php'
- 'public/admin/includes/sidebar.php'
- 'public/api/stats.php'
- 'public/assets/js/charts.js'
- 'public/assets/css/style.css'
- 'public/includes/header.php'
- 'public/includes/footer.php'

## Code Explanation

'Dashboard.php' reads admin statistics: number of users, hotels, rooms, reservations, open support tickets, reservation status totals, and confirmed revenue. The counts are optimized into one query.

'Payment.php' stores payment data for reservations. It can find an existing payment and create a new paid payment row.

'payment.php' checks that the reservation belongs to the logged-in user, is confirmed, and is not already paid. It validates the card number enough for this school project, stores the payment, regenerates the invoice, and redirects to the dashboard.

'invoice.php' protects invoice download. It checks that the reservation exists, the current user owns it or is admin, the reservation is confirmed, and payment status is paid. Then it calls 'InvoiceService'.

'InvoiceService.php' creates the PDF invoice. It writes customer information, hotel and room details, stay dates, number of nights, price, total due, payment method, card last four digits, and cancellation policy.

'config/config.php' stores database constants. 'Database.php' creates the PDO connection. 'bootstrap.php' loads all classes and starts the session.

'schema.sql' creates all tables: users, hotels, rooms, reservations, payments, and support tickets. It also inserts seed data.

'admin/index.php' shows dashboard cards and a reservation status chart. 'charts.js' draws that chart with canvas.

'style.css', 'header.php', and 'footer.php' create a consistent layout and design for all pages.

## How To Answer The Teacher

If the teacher asks "How does payment work?", answer:

"The user can only pay after the admin confirms the reservation. The payment page checks the reservation owner, status, and existing payment. Then it stores payment information and regenerates the invoice."

If the teacher asks "How is the invoice generated?", answer:

"The invoice page checks permissions and payment status. Then 'InvoiceService' builds a PDF file with reservation and payment information and stores it in 'public/invoices/'."

If the teacher asks "How does the admin dashboard get statistics?", answer:

"The Dashboard model sends SQL queries to count users, hotels, rooms, reservations, open support tickets, reservation statuses, and confirmed revenue."

If the teacher asks "How do we run the project?", answer:

"Start Apache and MySQL in XAMPP, import 'database/schema.sql' into MySQL, make sure 'config/config.php' has the correct database settings, then open 'http://localhost/hotel-management/public' in the browser."

## Complete File Description

### Config And Startup

- 'config/config.php': Stores database constants.
- 'config/Database.php': Creates the PDO connection.
- 'app/bootstrap.php': Loads the whole backend and starts the app.

### Core

- 'app/Core/Session.php': Session and flash message helper.
- 'app/Core/helpers.php': Shared helpers for URL, escaping, dates, money, photos, galleries, and room availability.

### Models

- 'app/Models/User.php': User database logic.
- 'app/Models/Hotel.php': Hotel database logic.
- 'app/Models/Room.php': Room database logic with hotel and reservation availability joins.
- 'app/Models/Reservation.php': Reservation database logic.
- 'app/Models/Payment.php': Payment database logic.
- 'app/Models/SupportTicket.php': Customer service ticket logic.
- 'app/Models/Dashboard.php': Admin statistics logic.

### Services

- 'app/Services/Auth.php': Login, register, logout, role checks.
- 'app/Services/InvoiceService.php': PDF invoice generator.

### Public Pages

- 'public/index.php': Home page.
- 'public/rooms.php': Public hotel and room catalog.
- 'public/reserve.php': Reservation form for one room.
- 'public/dashboard.php': User account dashboard.
- 'public/payment.php': Payment form.
- 'public/invoice.php': Invoice download controller.
- 'public/login.php': Login page.
- 'public/register.php': Register page.
- 'public/logout.php': Logout controller.

## Admin Pages

- 'public/admin/index.php': Admin dashboard.
- 'public/admin/hotels.php': Manage hotels.
- 'public/admin/rooms.php': Manage rooms.
- 'public/admin/reservations.php': Manage reservations.
- 'public/admin/users.php': Manage users.
- 'public/admin/customer-service.php': Manage customer service replies.
- 'public/admin/includes/sidebar.php': Admin navigation menu.

## API

- 'public/api/response.php': JSON helper functions.
- 'public/api/auth.php': Auth API.
- 'public/api/rooms.php': Rooms API.
- 'public/api/reservations.php': Reservations API.
- 'public/api/stats.php': Admin stats API.

## Assets

- 'public/assets/css/style.css': Custom styling.
- 'public/assets/js/rooms.js': Room catalog interactions.
- 'public/assets/js/reserve.js': Reservation page interactions.
- 'public/assets/js/charts.js': Admin chart drawing.

## Database And Docs

- 'database/schema.sql': Database creation and starter data.
- 'README.md': Project overview and setup.
- 'docs/API.md': API documentation.

## Short Presentation Plan

1. Student 1 starts with login/register and access control.
2. Student 2 explains hotels, rooms, catalog, photos, and availability.
3. Student 3 explains reservations, dashboard, support, and reservation API.
4. Student 4 explains admin, payment, invoice PDF, database, and final run steps.

## Final Teacher Summary

"This is an OOP PHP hotel management system. The public side lets users browse hotels, reserve rooms, pay confirmed reservations, download invoices, and ask support questions. The admin side manages users, hotels, rooms, reservations, support replies, and statistics. The backend uses PHP classes, PDO, MySQL, sessions, and helper functions. JavaScript is used only for interface improvements, while all important validation happens in PHP."